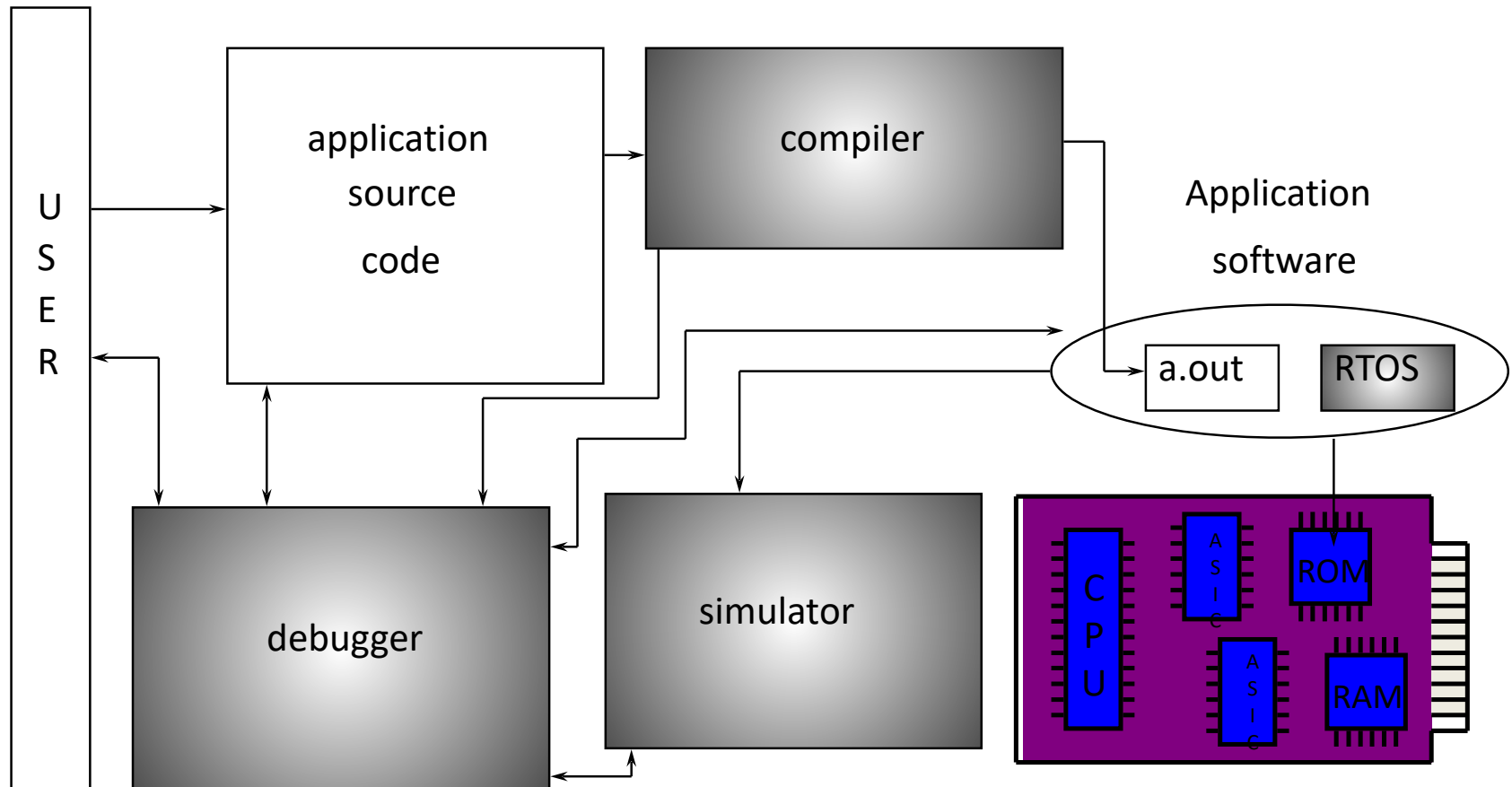


Software Design in SOC

Embedded Software Tools

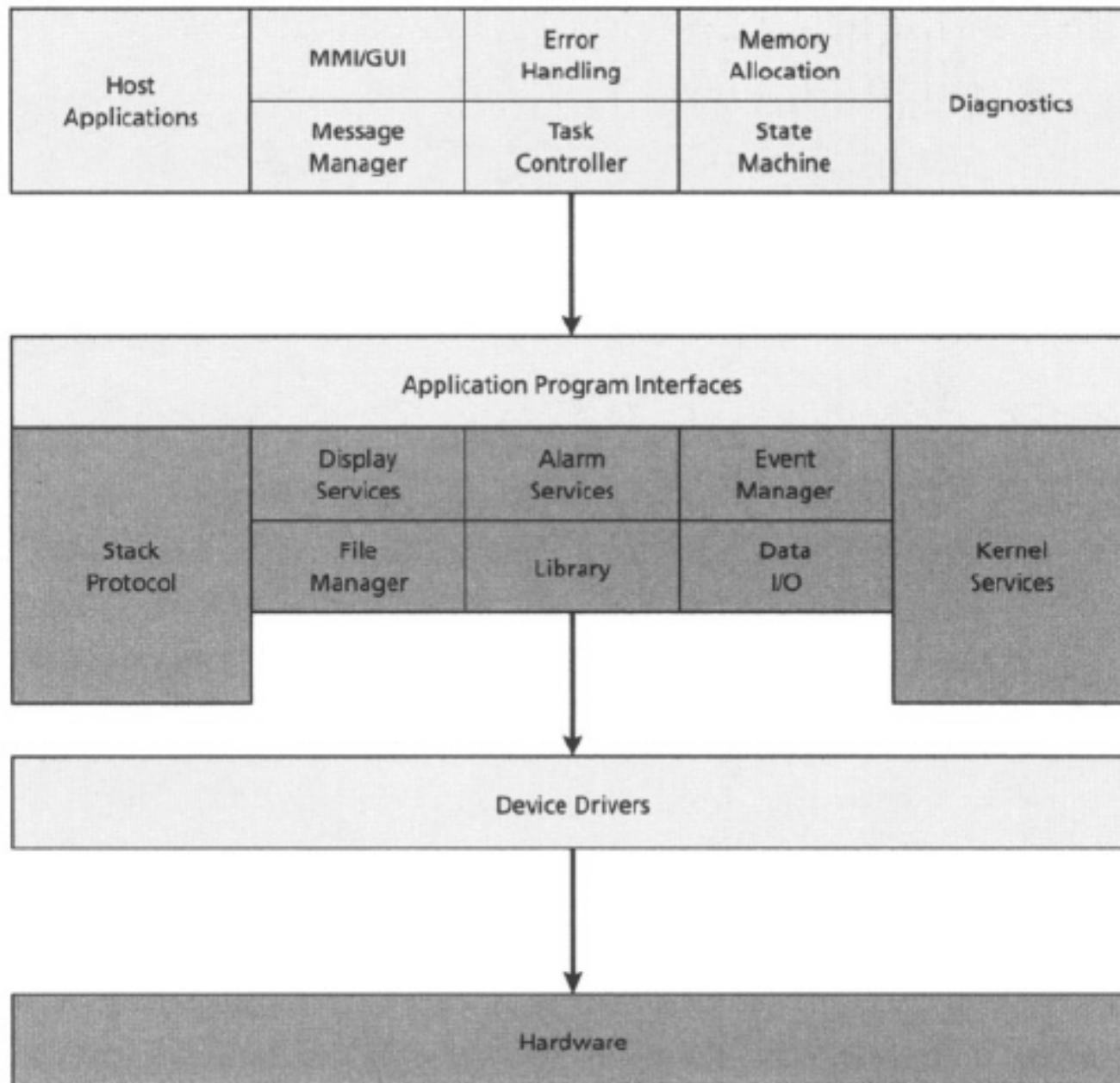


Components of a Software Development Environment

- **Critical components of a software development environment:**
 - **Compiler** for efficient code generation, predictable execution times of tasks and processes e.g. gcc, g++, javac.
 - **Interfaces** for generation of device drivers for external units and between hardware and software components.
 - **Embedded operating systems** for memory management, process management, I/O, synchronization, communication.

The Architecture of Embedded Software

- Embedded software has a layered architecture.
- Device drivers, which provide the basic hardware/software interfaces to specialized peripherals lying outside the processor, are closest to the hardware layer.
- Above it is the RTOS layer. It contains the RTOS kernel, which offers basic services such as *task scheduling* and *inter-task messaging*. This layer also contains the *communications protocol*, which makes the communications between the application layer and the hardware layer, management to handle hardware interrupts, and external IO stream management for files other data sources, displays, keyboards, and other devices.
- Higher-level system functions are made available to user applications and system diagnostics via Application Program Interfaces (API).



Job of Device Driver

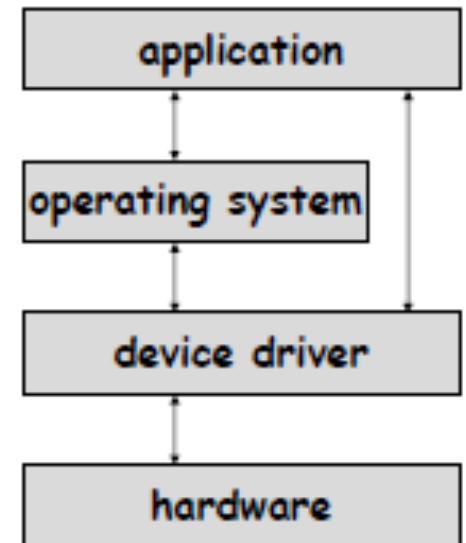
- **Initialization**

- register with operating system
- initialize interrupt service
- allocate memory
- initialize hardware

- **Data access**

- data exchange between application and hardware (e.g. memory mapped, DMA)
- processing of protocol layers

- **Interrupt service**



Issues involved in SOC software Development

- One of characteristics of embedded software is that it is heavily **dependent on the underlying hardware**.
- The reason of the dependency is that embedded software needs to be designed in an application-specific way.
- **To reduce the system design cost, e.g. code size, energy consumption, etc., embedded software needs to be optimized by exploiting the characteristics of underlying hardware.**
- Sometime when SoC designers need to implement a functionality that is not implemented by hardware blocks in the platform, they need to implement it in software.

Issues involved in SOC software Development

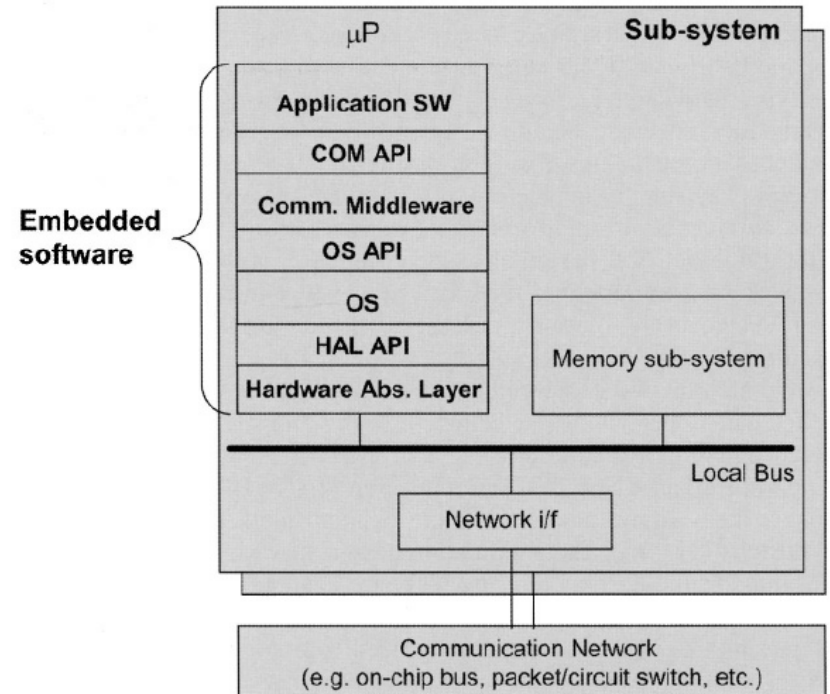
- To understand embedded software design for SoC, one needs to know **current issues in embedded software design**.
- These issues can be classified into two parts:
 - **Software reuse for SoC integration**
 - **Architecture-specific software optimization.**

Software reuse for SOC integration

- Due to the increased complexity of embedded software design, the **design cycle** of embedded software **is becoming the bottleneck to reduce time-to-market.**
- To shorten the design cycle, embedded software needs to be reused over several SoC designs. However, the hardware dependency of embedded software makes software reuse very difficult.
- **A general solution to resolve this software reuse problem is to have a *multi-layer architecture* for embedded software.**

Software Architecture in SOC

- An SOC consists of sub-systems connected with each other via a communication network.
- Within each sub-system, embedded software consists of several layers:
 - Application software,
 - Communication middleware (e.g., message passing interface),
 - Operating system (OS), and
 - Hardware abstraction layer (HAL).



Multi Layer Software Architecture in SOC

Cont...

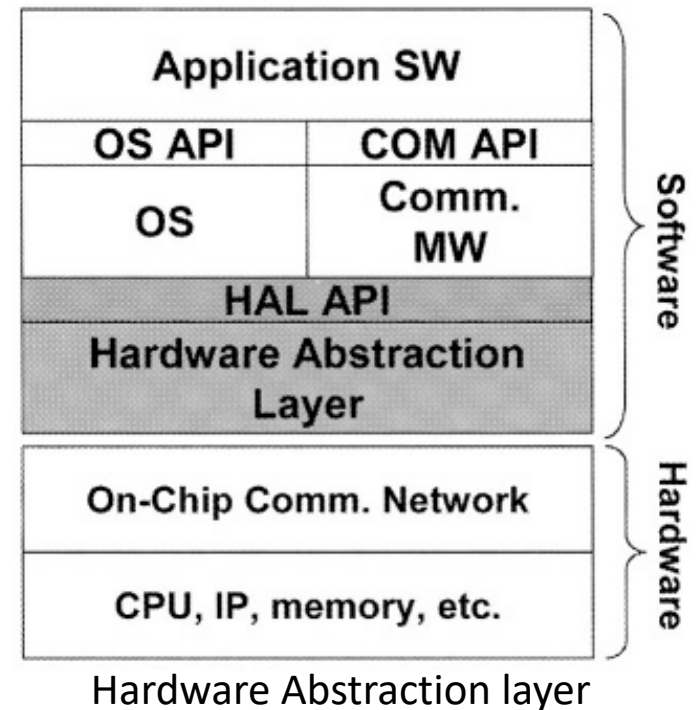
- **In the architecture, each layer uses an abstraction of the underlying ones.** For instance, the OS layer is seen by upper layers (communication middleware and application layers) as an abstraction of the underlying architecture, in the form of OS API (application programming interface), while hiding the details of OS and HAL implementation and those of the hardware architecture.
- **Embedded software reuse can be done at each layer.** For instance, we can reuse an RTOS as a software component. We can also think about finer granularity of software component, e.g. task scheduler, interrupt service routine, memory management routine, inter-process communication routine, etc.

Hardware Abstraction Layer (HAL)

- HAL is defined as all the software that is directly dependent on the underlying HW.
- The examples of HAL include boot code, context switch code, codes for configuration and access to HW resources, e.g. MMU, on chip bus, bus bridge, timer, etc.
- In real HAL usage, a practical definition of HAL can be done by the designer (for a given HW architecture), by OS vendors.

Cont...

- An HAL API gives to the upper layer SW, i.e. operating system (OS) and application SW an abstraction of underlying HW architecture, i.e. processor local architecture.
- Though HAL is an abstraction of HW architecture, since it has been mostly used by OS vendors and each OS vendor defines its own HAL, most of HALs are also OS dependent.
- In the case of OS dependent HAL, it is often called board support package (BSP).



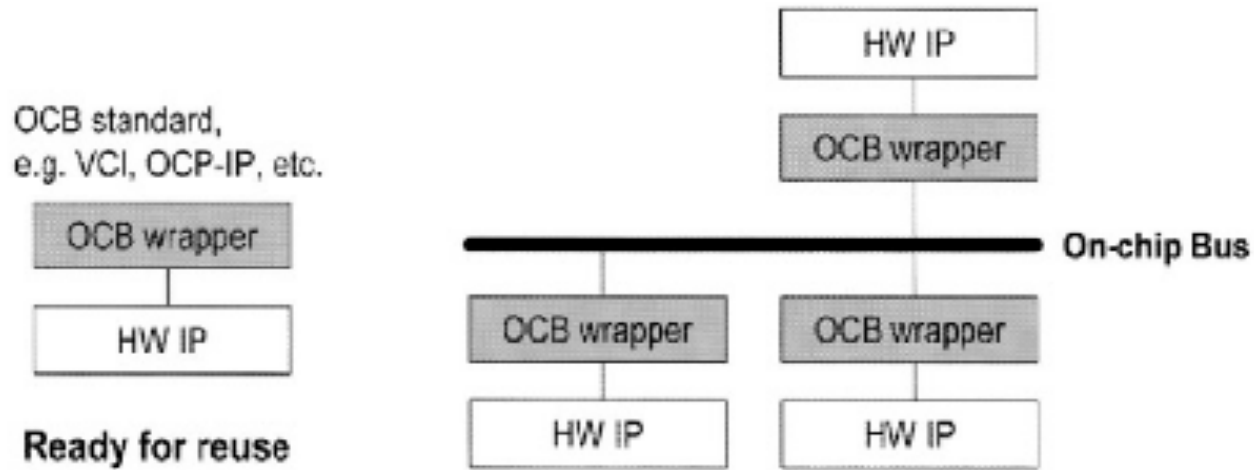
HAL for SOC design

- In the context of SOC design, HAL keeps still the original role of enabling the portability of upper layer SW.
- However, in SOC design, the portability impacts on the design productivity in two ways: SW reuse and concurrent HW and SW design.
- **Portability enables to port the SW on different HW architectures.** In terms of design reuse, the portability enables to reuse the SW from one SoC design to another.
- Easy porting of OS and application SW means easy reuse of OS and application SW over different HW architectures that support the same HAL API. Therefore, it can reduce the design efforts otherwise necessary to adapt the SW on the new HW architecture.

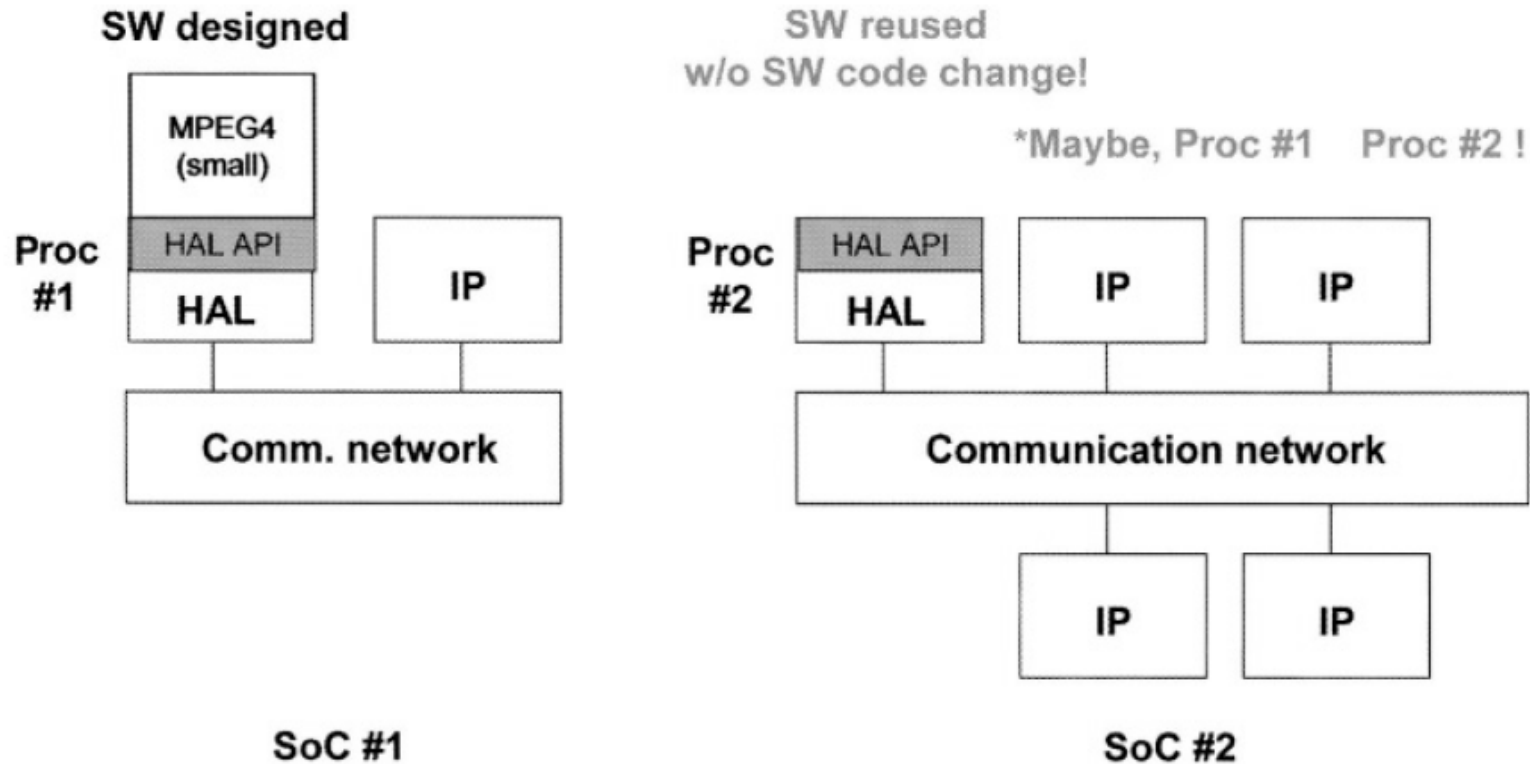
API's

- An Application Programming Interface (API) is a set of commands, functions and protocols which programmers can use when building a software.
- It allows the programmers to use predefined functions to interact with systems, instead of writing them from scratch.
- **Characteristics of good API**
 - Easy to learn and use, Hard to misuse.
 - Easy to read and maintain code that uses it.
 - Is programming language neutral.
 - Sufficiently powerful to support all computational requirements.

Design Reuse in Hardware



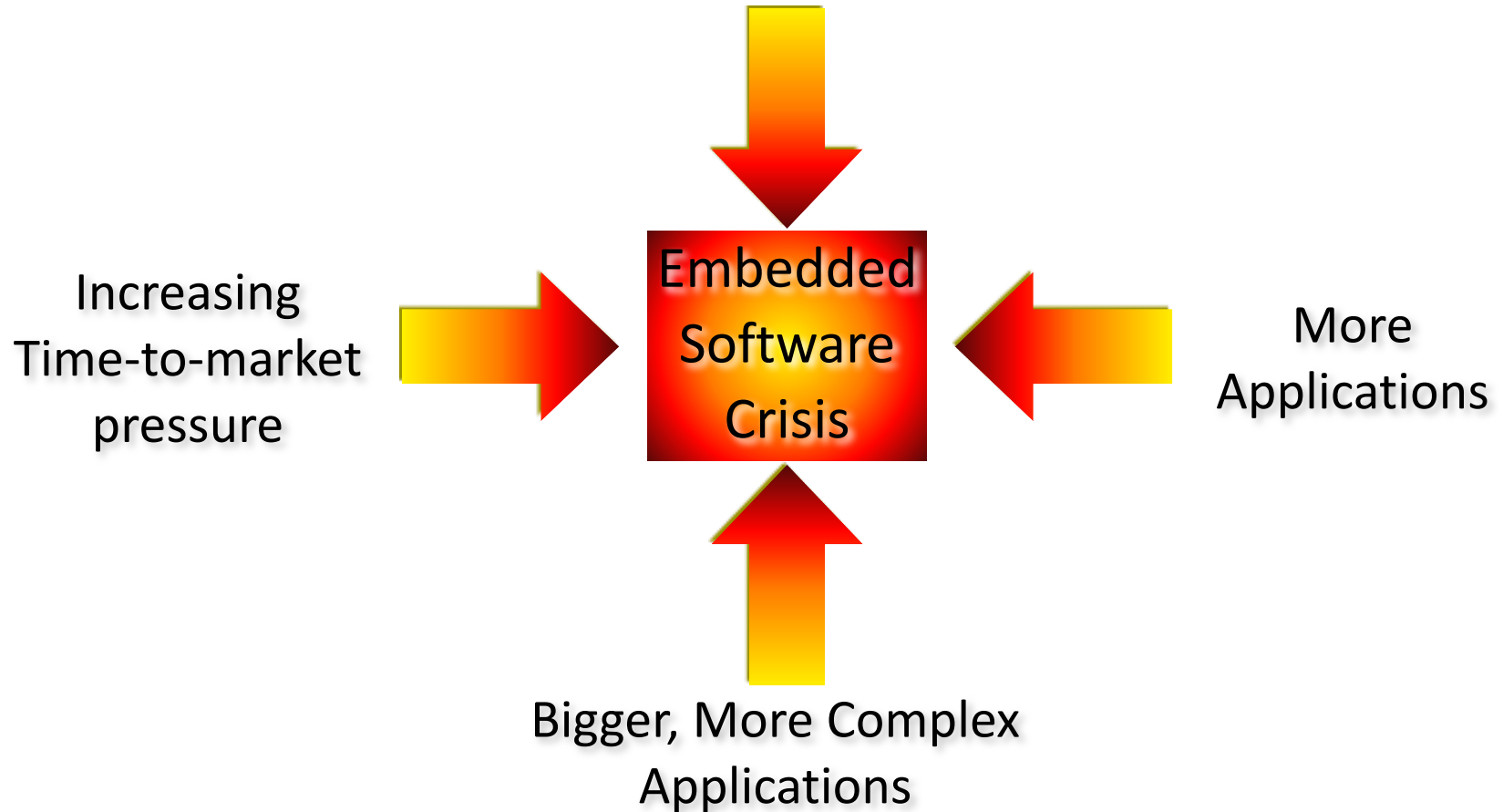
Design reuse in software



Embedded Software Crisis

Cheaper, more powerful

Microprocessors



Improving Embedded Software

- The hardware/software definition, trade-offs, partitioning, and modeling are done at the system level.
- The software architecture is an integral part of the application-oriented integration platform.
- The software architecture is carefully layered to minimize porting to new processors and RTOSs.
- Application-specific RTOSs are defined as part of the platform, when appropriate.
- Software is structured to maximize reusing software VCs.
- Further develop standards, such as the VC Interface, to help automate device driver development.

Embedded Software Development: Issues and Solution

Issue	Recommended Solution
What is preventing the emergence of an embedded software industry?	Rapid proliferation of hardware platforms (various embedded processors) and a profusion of RTOSs means that most tentative software VCs must be tediously ported and reoptimized for each processor-RTOS combination. This is especially true with assembly code, for example, in various DSP applications.
What are the trends in RTOS development?	Homogenization, as might emerge with new versions of Microsoft Windows CE, which are hard real-time. Differentiation—variety of RTOSs, new microkernels, new ideas for automated configuration and generation.
How is software ported to a new processor and RTOS?	To be efficient, software needs better APIs and layering to isolate processor and RTOS dependencies. Standardized APIs for device drivers, scheduling, and messaging services also need to be offered. Currently, this still involves extensive manual development and testing efforts.
How do I simplify generating device drivers?	There are new approaches to template-driven device driver synthesis for at least some classes of processors and devices.
What is the current hardware/software co-design and co-verification practice?	Very ad-hoc. Some emerging co-simulation tools and practices, but little standardization. Often little understanding of which techniques are available and how to use them.
How do I handle verification?	The method of orthogonalizing verification concerns. Cut the verification problem into layers and use various abstraction levels and models, and co-simulation techniques at different levels. Also reduce test cases as one goes down the verification stack by using slices of the upper test cases or targeted special-purpose tests.

System Integration Issues in SOC

- Analog IP reuse may not be straightforward due to changes in parameters that result in design issues. Some of these details are listed below:
- **Technology node change/migration:** This brings in a new set of challenges. The same analog IP may not necessarily work in a lower technology node due to the following issues
 - **Voltage scaling impacts the signal-to-noise ratio and signal propagation time.**
 - **Interconnect pitch scaling:** This results in changes in parasitics of interconnect and resulting signal integrity (SI) effects which can be a critical problem when integrating analog IP. For instance, the performance of a Low Noise Amplifier (LNA) can be extremely sensitive to the RLC parasitics on the input interconnect.
 - **Device performance changes impacting the interface noise.** A change in clock frequency may lead to different noise-coupling issues in the analog IP.

System Integration Issues in SOC

- **Requirement change:** Specification change even at the same technology node necessitates design changes impacting the functionality bringing in issues with design reuse to meet exactly same design performance requirements.
- **Foundry change:** It is also not very uncommon to observe different analog behavior for the same analog IP at a common technology node due to difference in process technology of the foundries.
- **Changes in EDA tools:** Different EDA tools may have different device model leading to different simulation results.